

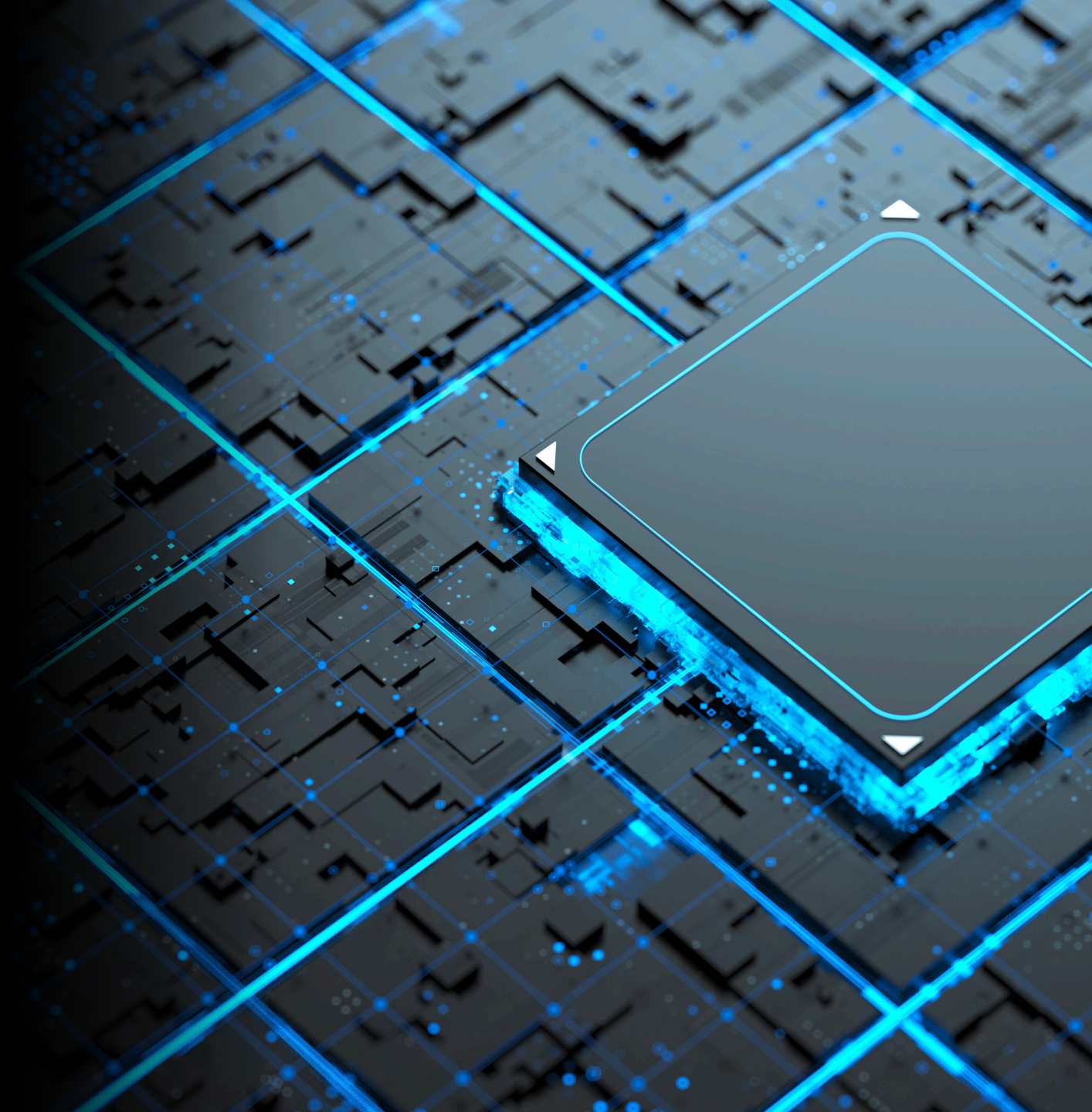
Platform Engineering

Internal Developer Platform

Bjørn Kristian Punsvik
2026-09-09



slides @ github.com/bjornkpu/talks



**Utviklerne dine er blant de
dyreste ressursene du har.
Hva gjør de når de ikke
skriver kode?**

**De beste ops-folkene dine
svarer på tickets. De løser
ikke arkitektur.**

**De beste utviklerne dine
bygger ikke produkt. De
vedlikeholder
infrastruktur.**



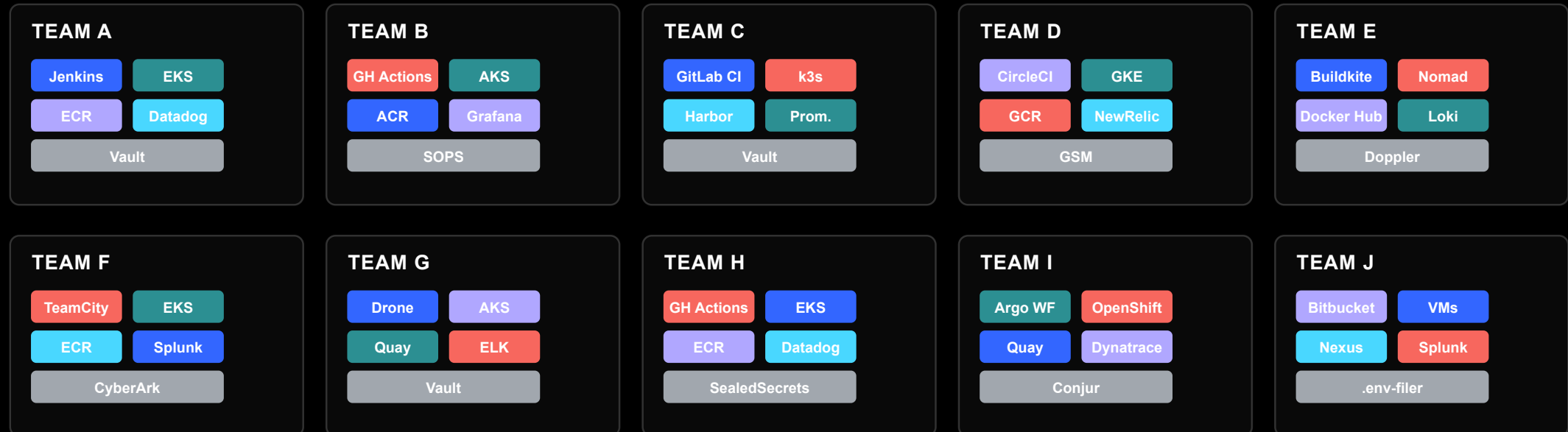
```
bk@softwareone:~$ whoami
```

```
-----  
Name:      Bjørn Kristian Punsvik  
Role:      Data & Software Engineer  
Position:   Senior Consultant  
Location:   Trondheim  
Uptime:     30 years  
Experience: 6 years  
Education:  B.Sc. Computer Engineering, NTNU
```

```
bk@softwareone:~$ ps -a
```

```
-----  
START      DURATION  PROJECT  
2017-08     3y        /ntnu/tihlde-drift  
2020-07     2y 3m     /enoco/eurora-cloud  
2023-06     1y 6m     /equinor/grc  
2024-12     3m        /equinor/fos  
2025-09     →        /enova/mimir  
2026-03     →        /enova/ai
```

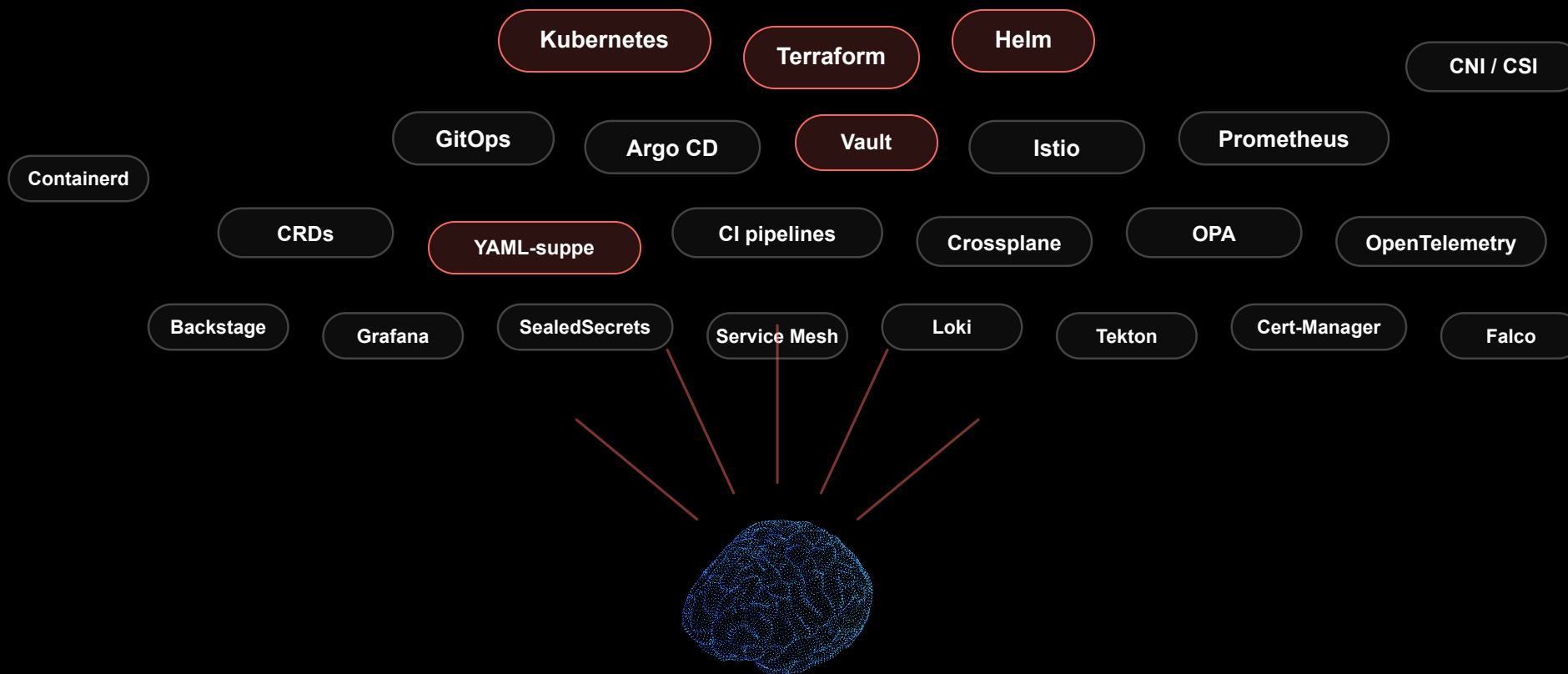
Slik ser det ut når den ikke er bygget med vilje



SECURITY & COMPLIANCE

Ett team. Ti stacker. Null oversikt.
"Hvordan er state-of-secrets på tvers?"

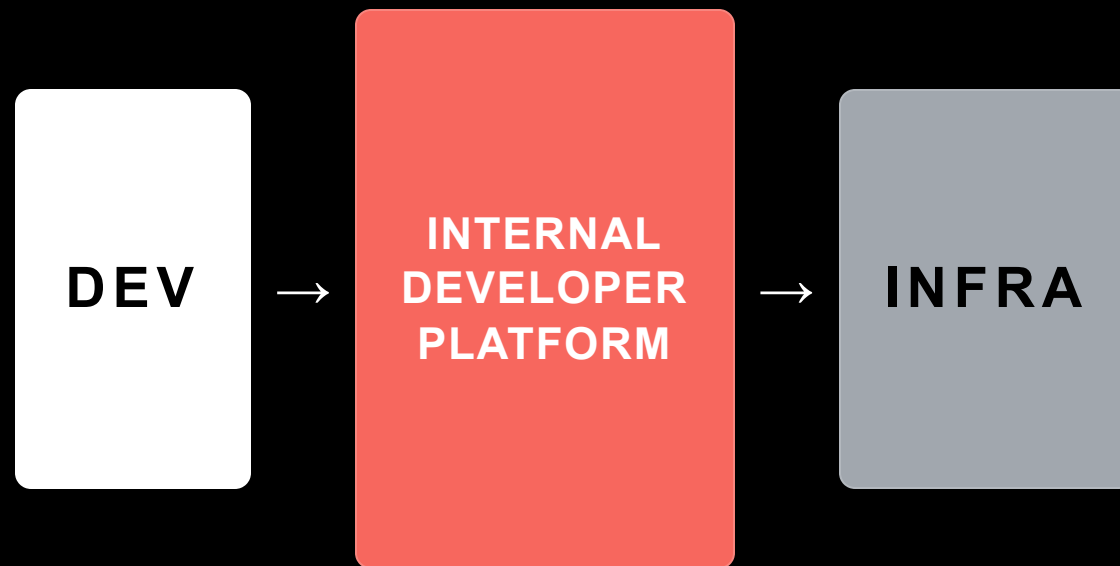
Cloud-native ble for mye for ett menneske



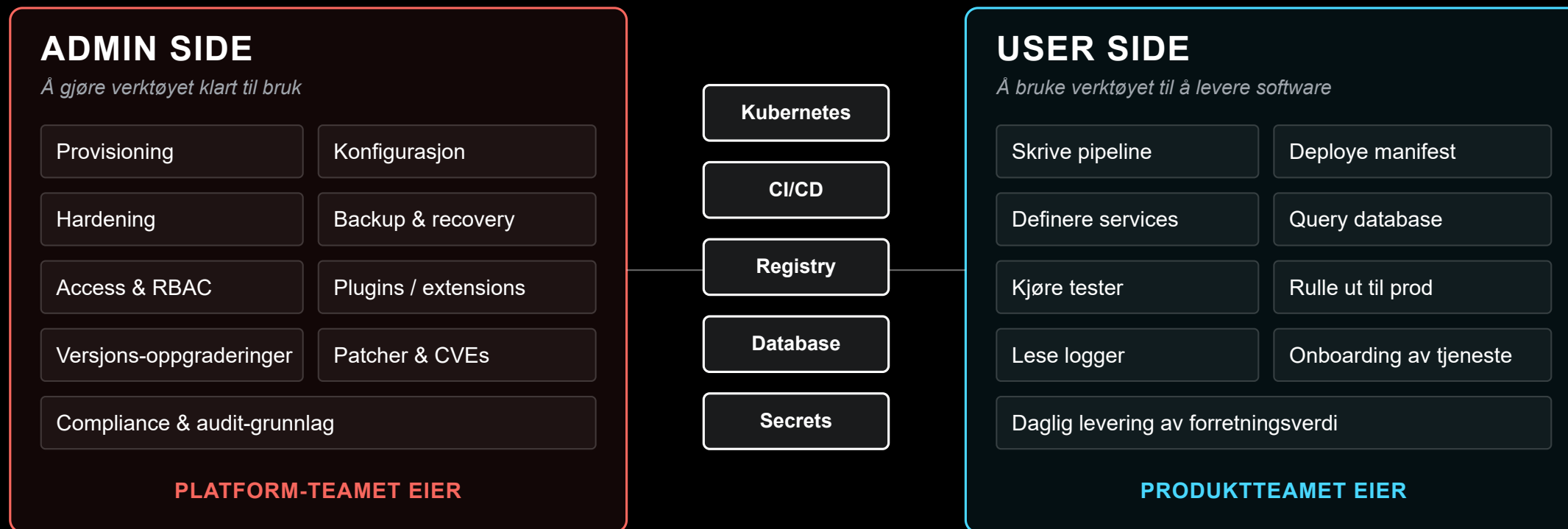
Platform Engineering

Disiplinen som bygger **golden paths** — så utviklerne får self-service på det de trenger, uten å vente på ops.

Et skreddersydd lag mellom utvikler og infra. Et **produkt**, ikke et prosjekt.



Hvert verktøy har to sider



Samme verktøy. To forskjellige fag. Egne sertifiseringer.
Når en utvikler skal kunne begge — det er der den kognitive lasten kommer fra.

Internal Developer Platform — en IDP

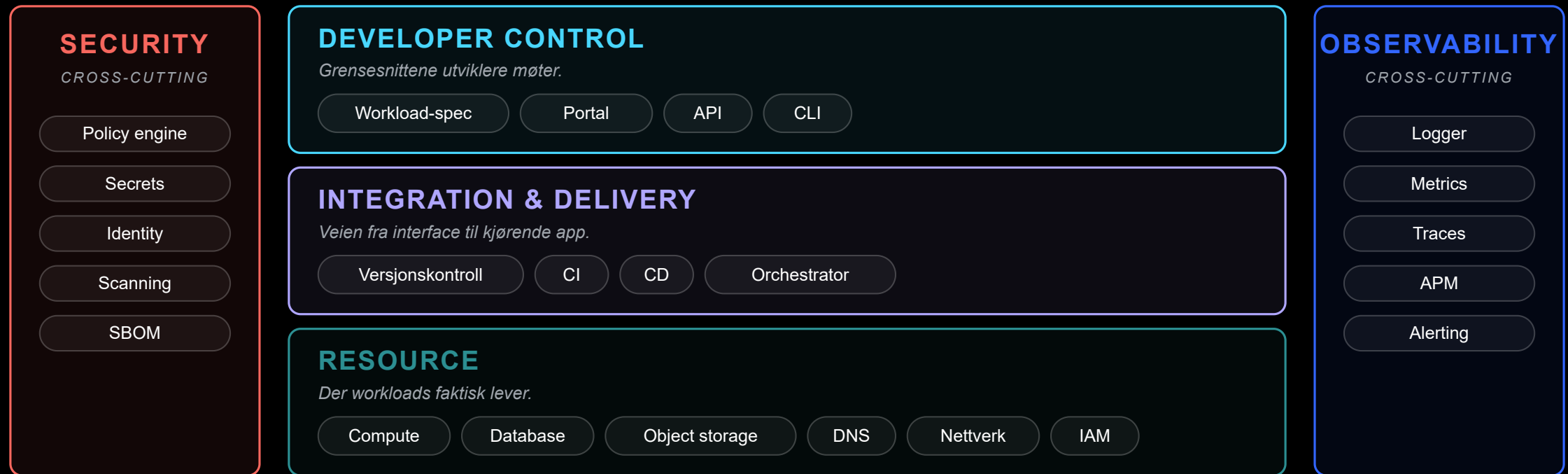
Det er

- Anbefalte veier for det utviklere gjør oftest
- Selvbetjening — ingen tickets
- Sikkerhet og best practice innebygd fra dag én
- Et **produkt**, ikke et prosjekt

Det er *ikke*

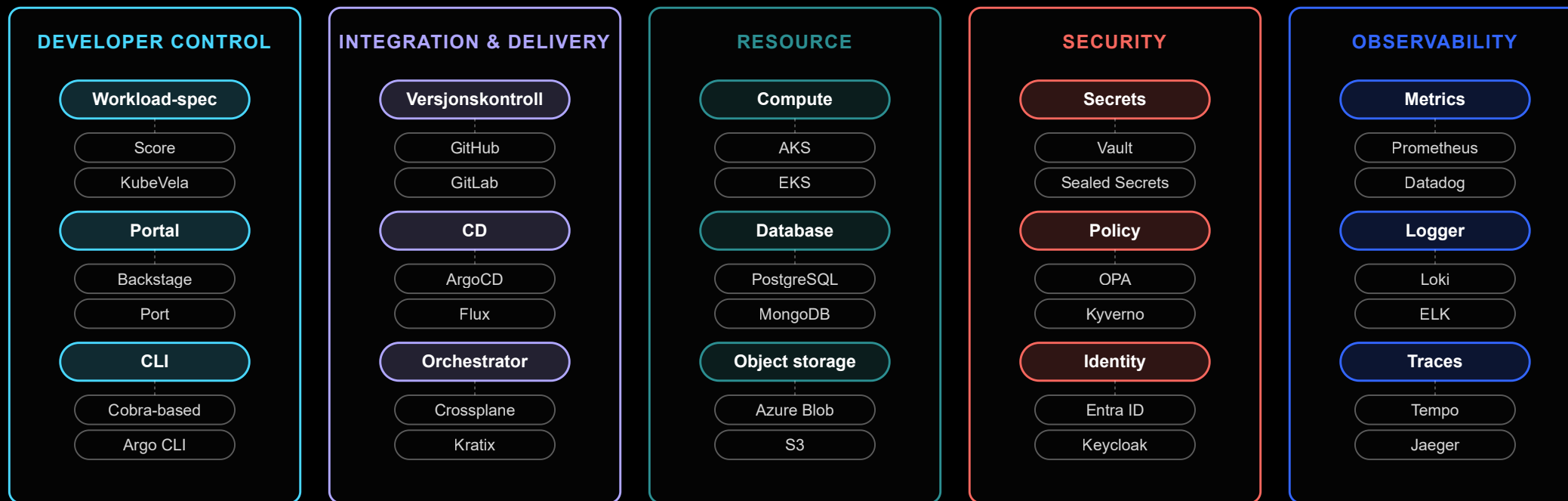
- En portal
- Et ticketssystem med penere UI
- Et Kubernetes-cluster noen dokumenterte
- En samling verktøy uten sammenheng

Anatomi av en IDP — fem planes



Tre stablede planes utviklere beveger seg gjennom. To tverrgående bånd som krysser alt.

Ports & adapters — én port, mange verktøy

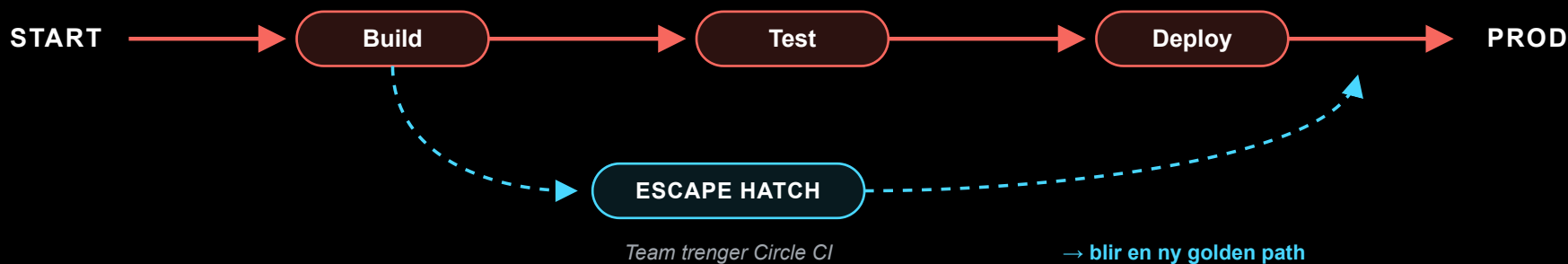


Hver port er et **behov** (versjonskontroll, secrets, metrics). Adapterne under er **valg** — utskiftbare verktøy. Når et team trenger et nytt adapter, vokser katalogen.

Golden paths er en paved road — ikke et gjerde

GOLDEN PATH

Anbefalt — ikke obligatorisk. Vi har testet den. Resten er vårt problem.



Veien er **anbefalt**, ikke obligatorisk. Den fungerer, dokumentasjonen finnes, vi har testet den. Det finnes alltid en escape hatch — og når et team trenger en, blir den en ny golden path.

En golden path er ikke en regel.

Det er et løfte:

"Gjør det slik — så er resten vårt problem."

Et produkt, ikke et prosjekt

SOM PROSJEKT

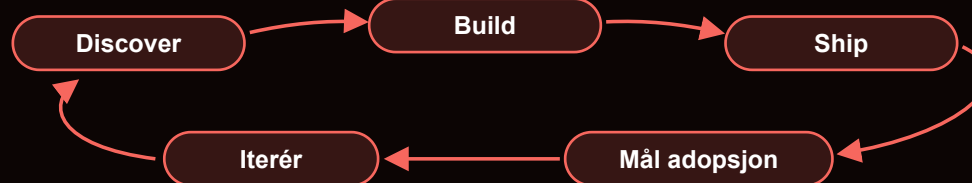
Bygges, leveres, ferdig — adopsjon = pålagt



→ Hyllevare med gode intensjoner

SOM PRODUKT

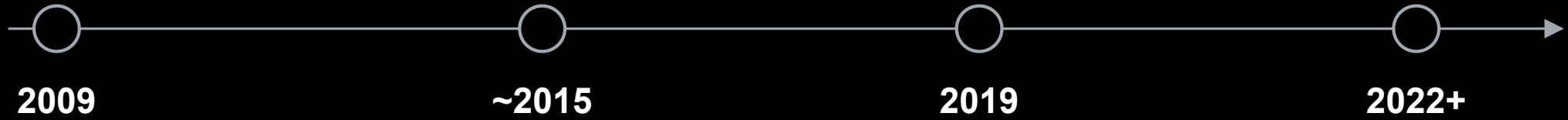
Utvikles kontinuerlig — utviklere er kunder, adopsjon er metrikken



→ Et produkt utviklerne velger frivillig

Plattformen leveres ikke ferdig. Den utvikles kontinuerlig — med utviklere som kunder og adopsjon som suksessmetrikk.

Hvordan kom vi hit?



Platform Engineering dukket ikke opp i et vakuum. Det er svaret på et problem som har bygget seg opp i over tjue år.

2009 — DevOps

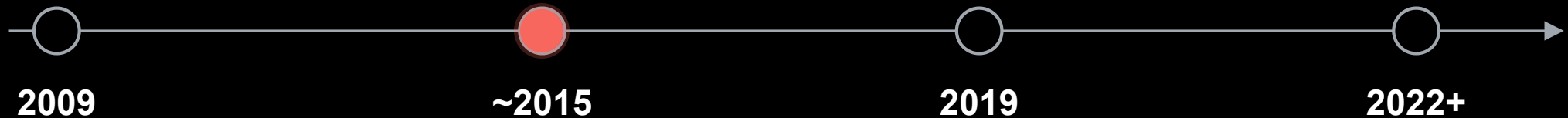


Patrick Debois ga navn til det alle visste: muren mellom dev og ops må ned.

Kultur. Felles ansvar. *"You build it, you run it."* Revolusjonerende.

Men så kom skyen.

~2015 — "You build it, you run it" blir for mye



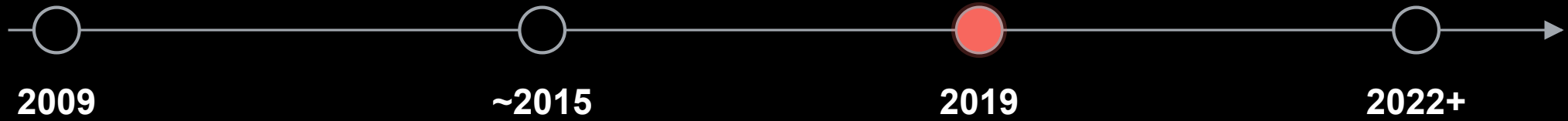
Cloud ga oss alt vi trengte — og hundre nye måter å skyte oss selv i foten på.

Kubernetes. Terraform. Helm. Service meshes. Hver utvikler forventes å være infraekspert.

Senior-devs havner i *shadow operations*. Ticket-køene vokser. Kognitiv last spiser dagen.

DevOps var ikke feil. Det ble bare for mye for ett menneske å bære.

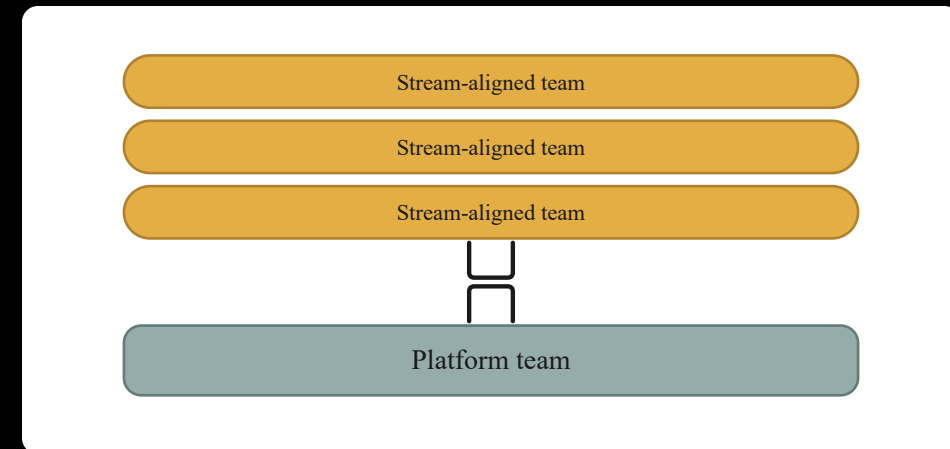
2019 — Team Topologies



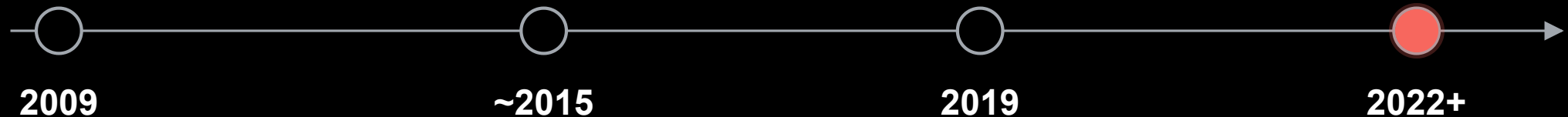
Matthew Skelton og **Manuel Pais** navngir fire teamtyper — Stream-aligned, Platform, **Enabling**, og Complicated-subsystem.

Platform-teamet leverer **produktet**. **Enabling-teamet** leverer **capability** — midlertidig, til andre team kan stå på egne ben.

Plattform som produkt. Utviklere er kunder. Grunnmuren for alt vi gjør i dag.



2022 — Platform Engineering tar form



Gartner: top 10 strategic tech trend for 2023.

platformengineering.org passerer 8 000 medlemmer i 2022 — over 270 000 nå.

IDP-en blir den konkrete artefakten — **produktet** plattform-teamet leverer.

Ikke DevOps på nytt. Ikke SRE i nye klær. En egen disiplin — med utvikleren som kunde.

Norske IDP-er — i drift



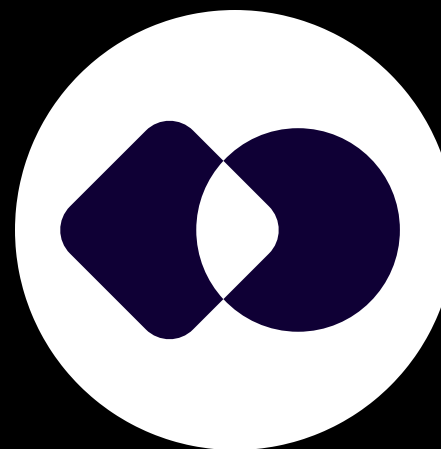
NAIS — NAV

~100 team, 1 600+ apper i produksjon. Åpen kildekode.



Radix — Equinor

AKS-PaaS, deklarativ via `radixconfig.yaml`. MIT-lisens. To produksjonsclustere + playground.



Platon — SIKT

AWS EKS + GitLab. Fellesplattform for kunnskapssektoren. Visjon: *"Norges beste utvikleropplevelse."*

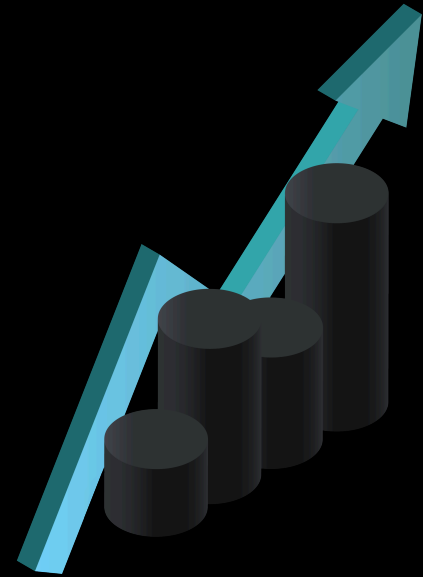


Njord — Enova

AKS + ArgoCD + Dapr. Lansert mars 2026 som svar på digital suverenitet — bygget for leverandøruavhengighet.

Verdien

For utvikler, ops, og virksomhet



Samme plattform — tre vinninger



UTVIKLER

🕒 Miljø på minutter, ikke dager

🧠 Mindre kognitiv last

🚀 Slipper å være infraekspert



OPS

📁 Ut av ticket-helvete

🛡️ Standardisering by design

🎯 Tid til strategisk arbeid



VIRKSOMHET

▶▶ Lavere lead time for change

📉 Lavere change failure rate

👥 Raskere onboarding av folk

Plattformen er **ikke en IT-kostnad**. Den er en investering i organisasjonens leveringskraft.

ROI er ikke en innkjøpskalkyle

"Vi kjøpte X. Sparte vi mer enn X?" — feil spørsmål.

Platform-ROI er adferdsendring i skala.

Tre regnestykker — illustrative

Tid frigjort

100 utviklere × 2 t/uke × 50 uker
× 1 500 kr/t

≈ 15 MNOK

Kapasitet frigjort — ikke penger spart.

Feil unngått

20 færre alvorlige hendelser ×
250 000 kr

≈ 5 MNOK

Direkte, målbar risikoreduksjon.

Hastighet

5 nye utviklere produktive på 1
uke i stedet for 6

≈ 1,5 MNOK

Onboarding-friksjon kuttet med
6×.

Hvordan starter du?

Uten å rive ned alt



Start med en Minimum Viable Platform



Lav innsats. Lav risiko. Reell læring. **Ikke produksjon — ikke ennå.** Du finner de ødelagte kontrollene mens stakes fortsatt er lave.

Møt teamene der de er

← HVOR DE ER I DAG

DIN IDEAL-TILSTAND →



Plattformen formes av virkeligheten i produktteamene — ikke av idealtilstanden din. Hjelp dem flytte seg inkrementelt. Versjon 1 må kanskje støtte litt av det gamle før den fortjener å utvikle seg bort fra det.

Feil måte å starte på

✗ Feil åpningstrekk

«Velg én CI/CD — og migrer alle.»

Løser ingen smerte teamene faktisk føler. Legger på arbeid.

Møtes med motstand.

✓ Riktig åpningstrekk

«Vi tar den verste delte smerten.»

K8s-drift, secrets, observability. Fjerner arbeid fra dag én.

Tjener credibility.

Standardisering kommer senere — først må du tjene retten til den.

Hva vi bygger i SoftwareOne

Vi har bygget **Canopy** — vår egen interne utviklingsplattform, på åpne standarder:

- **Kubernetes** — sky-agnostisk, ingen vendor lock-in
- **Argo CD** (Apps-of-Apps) — deklarativ, versjonert leveranse
- **Custom operatorer** — skreddersydd automasjon for plattform-workflows
- **OIDC** — føderert identitet på tvers av plattformen

PoC-en er ferdig. Nå bygger vi **produktlaget**: golden paths, dokumentasjon, feedback loops.



Enabling Team as a Service

Du trenger ikke ansette et komplett platform-team på dag én.

SoftwareOne kan fungere som et midlertidig enabling team — bygge MVP-en sammen med dere, etablere golden paths, og overlevere når deres eget platform-team er klart.

Callback til Team Topologies: et enabling team eksisterer for å gi andre team capabilities de mangler — og oppløses, eller flyttes, når jobben er gjort.



Etter pausen — to konkrete eksempler

Håvard — Kubernetes Operators

Hvordan reduserer vi kompleks infrastruktur til én utviklervennlig ressurs?

Live-bygg av en controller — operator-mønsteret i praksis.

Det er ikke et ferdig produkt — vi starter en **samtale**.

Marius — Canopy live

Fra onboarding av ny utvikler til app i prod.

Automatisk repo, CI-pipeline, og deploy via ArgoCD.

Slik feiler det — tre mønstre

Bare nytt skilt

Døp om ops-teamet til platform-team. Samme tickets, samme køer.

Samme kø, nytt navn.

Bygg for deg selv

Ex-ops bygger det *de* kjenner. Løser ops-problemer, ikke utviklerproblemer.

Et år arbeid, feil problem.

Portal ingen ba om

"Developer portals er hot."
Bygges uten brukerresearch.

Demo imponerte. Ingen logget inn igjen.

Tre feil. Én rotårsak: **bygget uten å snakke med utviklerne.**

*Hvis utviklerne dine bruker tid på noe annet
enn å skrive software som skaper verdi*

Da har du et plattformproblem.

Tre spørsmål å ta med hjem

1. Hvor lang tid tar det fra idé til kjørende tjeneste i prod?
2. Hvor mange tickets håndterer ops som ikke burde eksistert?
3. Hvor lenge før fem nye utviklere er produktive?

Svarene beskriver **plattformgapet deres**.



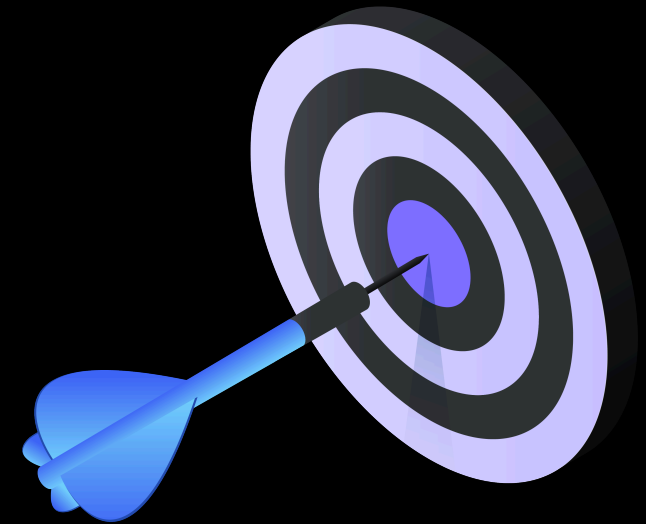
Målbildet

En organisasjon der utviklere skriver **software som skaper verdi** — ikke kjemper med infrastruktur under.

En plattform som er **et produkt**, ikke et prosjekt.

Raskere leveranser. Lavere risiko. Mindre teknisk gjeld.

Det er det Platform Engineering leverer.



Takk

Bjørn Kristian Punsvik



slides @ github.com/bjornkpu/talks

